Software Process Management

Lecture 5

Today's topic

- We have discussed about software processes and models to represent those processes
- Now let's discuss types of process models in this lecture



- We'll discuss the properties of these process models

After learning this topic

- You will be able to:
- Distinguish prescriptive and descriptive process models
- Explain the relationship between product and process
- Explain the prerequisites for applying prescriptive process models
- Name and explain different lifecycle process models

After learning this topic…

- Name and explain some widely used process standards
- Develop a process handbook for a specific target audience
- Support the deployment of a prescriptive process model to an organization

Process model types

- **Prescriptive Process Models**
- **Descriptive Process Models**



- Both types may exist in parallel in a software organization, often in several instances.

Process Models

- Prescriptive process models tell us how something should be done.
- Descriptive process models describe how something is done in reality.
- A **descriptive process model** is created by *observing the processes actually performed*, a **prescriptive process model** typically aims to address all relevant issues of developing a piece of software.

Prescriptive Process Models

- Prescriptive process models therefore often based on an ideal model of software development.
- For example, a set of best practices that should be applied in projects in order to yield their benefits.



- Challenge –
- Prescriptive Process Models lead to a number of conflicts.
- Human beings often tend to be somewhat reluctant in applying practices (or a process) perceived as boring, unnecessarily complicated, or laden with useless overhead.

Prescriptive Process Models

- Especially if the prescribed process demands work that does not seem to directly benefit the working person itself, or only little so, it is often hard to persuade this person to do work perceived as unnecessary.



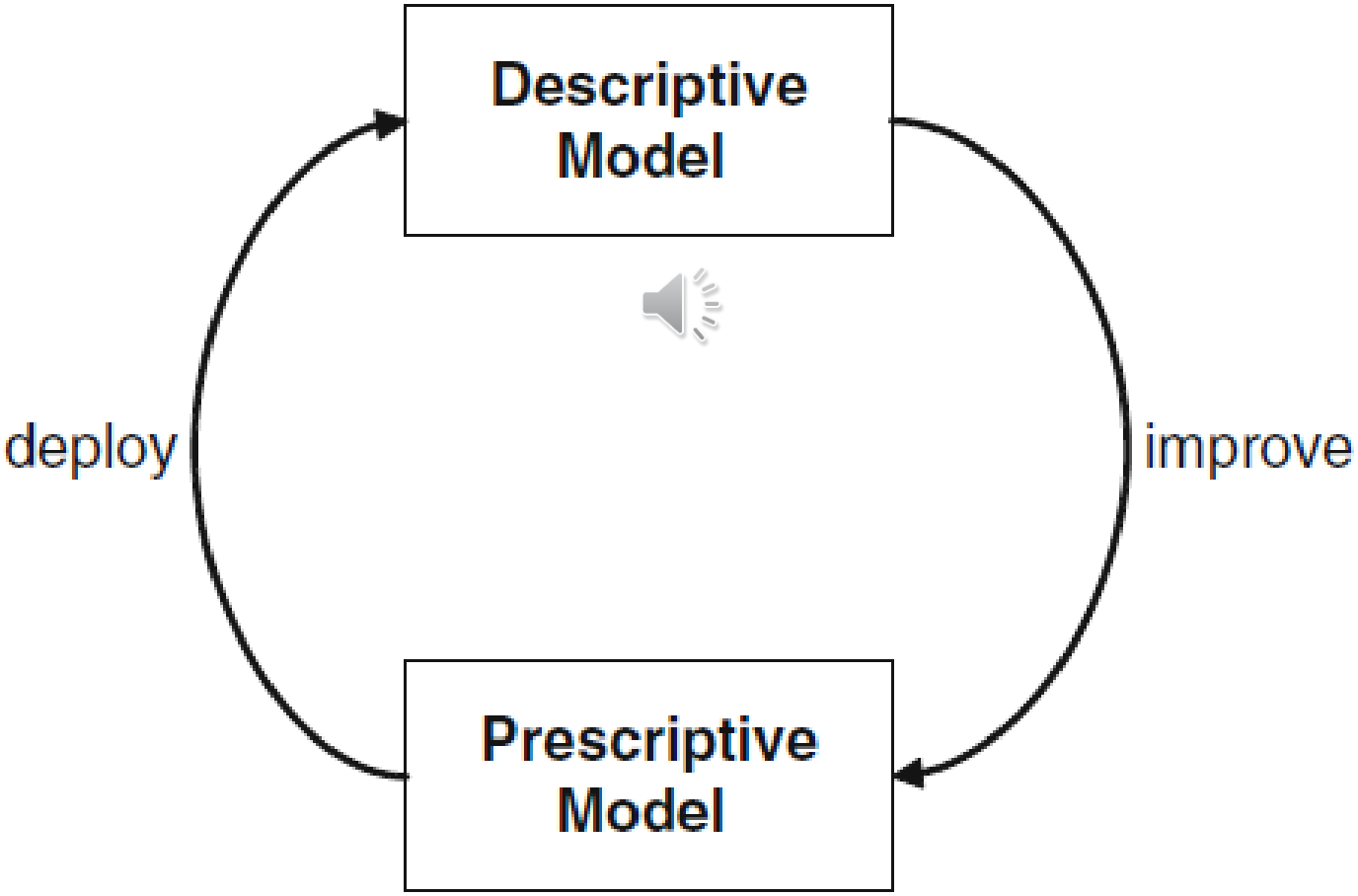
- The problem is not only to construct a prescriptive process model, but also to get people to follow it as intended.

- In fact, the latter is more difficult than the former, but often neglected.

The challenge

- A descriptive process model describes what people do every day—which means that it reflects current practices.
- A prescriptive model tells people to do (some) things differently, which means that they need to change their behaviour.
- Getting people to change their behaviour is one of the most difficult tasks in software engineering, so the successful deployment of a prescriptive process model is difficult, too:
- Deploying a prescriptive process model means changing people's behaviour!

The Relationship b/w DM and PM



In this relationship…

- Often, the current processes are modelled and transferred into a descriptive process model.
- Using this model, problems are identified and mitigated by changes to the process model, often using additional external knowledge such as best practices.
- This model then becomes a prescriptive process model, effectively instructing people to do things differently than before.
- When the changed process is fully integrated into people's daily work, the process model becomes descriptive again, and another improvement cycle can start.

The Product and Process

- The problems addressed by process changes are typically properties of the software product or the project that produces or modifies the product.
- E.g. the product might not be fulfilling its requirements or getting finished later than planned.
- When a problem was detected with the product, one should aim at solving this problem by changing the process.
- This is feasible because of the relationship between the process and quality aspects of the products.

- This relationship can be described according to the process–product relationship model from Rombach

$$Q \approx f\ (\text{Process}, \text{Context})$$

- Q means some quality aspect of the product, for example, reliability or cost of construction.
- This quality aspect is determined by a function of both process and context, such as a model-based testing (process) in the automotive domain, where the experience level of the testers is low and the programming language is C (context).

- So, in order to improve the reliability of a specific product, changing the context might be an option, for example, by improving tester experience.

- If this cannot be done, it might be possible to adjust the process, in order to cope better with inexperienced testers.

Process and Goal

- Besides quality aspects of the produced or modified product, other important properties of a project can be affected by processes.
- Such properties may be, for instance, the duration or effort consumption of a process.
- In addition, services can be considered as the object being developed, maintained, or operated by a process.
- Therefore, the function f can be described in more general terms as

$$\text{Goal} \approx f\ (\text{Process},\ \text{Context})$$

Process and Goal

- The goal can be any relevant product, service, or project property that can be affected by the process in the specific context.

- Function f is an empirical relationship
- Because the methods and processes are not yet well understood and thus not completely manageable.

- Well understood means that the effects of methods and processes on goal properties (such as product quality, cost, time) in varying contexts (e.g., different lifecycle models, personnel experience, or tool support) are known.

- Manageable means that for a given project context, processes can be chosen so that they guarantee the expected effects (such as product qualities).

- f is nondeterministic and can only be determined empirically!

- This implies that process and context must be described as precisely as possible, and that the respective goal properties need to be defined so that they are measurable.

When to get results

- The impact of changes to the process or context can be determined through empirical studies of different types:
  - qualitative or quantitative studies,
  - controlled experiments or case studies,
  - real studies or simulations
- The results can be used to create prescriptive process models proven to be beneficial in certain contexts
  - → guarantee specific product qualities
- Hence, process changes only rarely result in major product quality improvement within a short period of time
- Process improvement is a mid- to long-term effort.
- Process changes will yield most of their benefits in the long run.

Prerequisites

- To apply any prescriptive process model, a number of prerequisites should be fulfilled.
- If one or more prerequisites are not fulfilled, the effects of applying the prescriptive process model may be unpredictable.
- It is more likely that it does not work as planned, leading to worse results than planned, possibly even worse than before it was applied
- Which means that the new prescriptive model is likely to be rejected

Prerequisites

- 1) **The scope of validity should be known**
- It should be known which context(s) and goal(s) the process model is valid for.
- Context comprises factors such as
- Domain characteristics (e.g., aerospace: highly safety critical, with products being used for decades),
- Organizational characteristics (e.g., large organizations with low employee turnover),
- Project characteristics (e.g., long-term projects with distributed development).

- Goals comprise, for example, the reduction of cycle time or the improvement of code quality.
- A prescriptive model may have been applied successfully in a large aerospace organization with long-term projects and low employee turnover, leading to a significant reduction in cycle time.
- Applying the same model in a small Web application development organization with short-term projects and high employee turnover and the goal of improving code quality is likely to not achieve this goal, because the model was never intended for this context and goal.

Prerequisites

- 2) **The impact of a process should be known**.
- It should also be known what the effects of the process described by the prescriptive process model are for a specific context.
- Effects may be, for example, the reduction of cycle time by 30% or the reduction of design defects by 20%.
- The specific context may be a large aerospace organization with long-term and distributed projects and low employee turnover.

- Applying the same process as prescribed by this specific process model in the same or a similar context is likely to have a similar impact.
- If the impact of a prescriptive process model is unknown, applying it leads to unpredictable results.
- Knowledge of this kind is usually specific for an application domain or organization and thus must be obtained in the respective setting.

Prerequisites

- **3) The degree of confidence should be known.**

- It should be known to what degree of confidence the process described by the prescriptive process model has been evaluated in a specific context.

- For example, the reports on the impact of a specific process may stem from a single application in a specific context—in this case, the degree of confidence is rather low, since other factors may have played a significant role, but were not recorded.

- On the other hand, if a specific process has been applied in a multitude of similar contexts, with the desired effects reported in all or most cases, the degree of confidence is rather high.

- Because it is very unlikely that other factors were responsible, but not recorded, in all the success cases.

Prerequisites

- **4) The process should be flexible to be tailored when needed.**
- Finally, it should be possible to adapt the process described by the prescriptive process model to specific project goals and contexts.



- Since the context for which a process model has been developed rarely presents a perfect match for the context it is supposed to be applied in next, and since goals may also differ, it should be possible to adapt the process and its corresponding model to the new context.

- For example, if in the original context, employee turnover was low, but in the new context, it is high, the process must be adapted to cope with this.
  - e.g., by increasing the (written) documentation of project information.
- Obviously, there must be sufficient knowledge about the effects of the process changes in order to retain the validity of the process model for the changed context.

Prescriptive Process Model Classes

- **Lifecycle process models** capture the complete lifecycle of a software product.

- Typically, they abstract from a number of details, and instead provide a broader view on the process (focus on “what,” not on “how”).

- **Engineering process models** describe (possibly in very much detail) a fraction of the complete software lifecycle process, for example a specific type of inspection.



- Engineering process models can be very detailed, often not only describing “what” to do, but also explaining “how” to do it.

Lifecycle Process Models

- Lifecycle process models typically cover the complete lifecycle of a software product or a large fraction of it.
- Which in turn generally leads to a high level of abstraction.
- Therefore, lifecycle process models typically tell the software engineer “what” to do, and in what sequence, but not “how” to do it.

- They usually do not recommend (or prescribe) any specific method or technique, but rather demand that something needs to be done “effectively,” meaning that whatever method is applied, it must yield the expected result(s).

- For example, a lifecycle process model may demand that traceability be established between requirements and components, but it will not prescribe how this should be achieved (with or without a tool, how many traceability links are expected, etc.)
- This can (and must) be determined by the process engineers at the respective organization.

Example LC Process Models

- The Waterfall Model
- It was first formally described by Winston Royce in 1970
- The idea behind the waterfall model is the sequential creation of products on different levels of abstraction (e.g., precede code by design, precede design by requirements) and integration in reverse direction.

- The strict sequence can be weakened by iterative interactions – continuous arrows
- Jumping between steps – dotted arrows

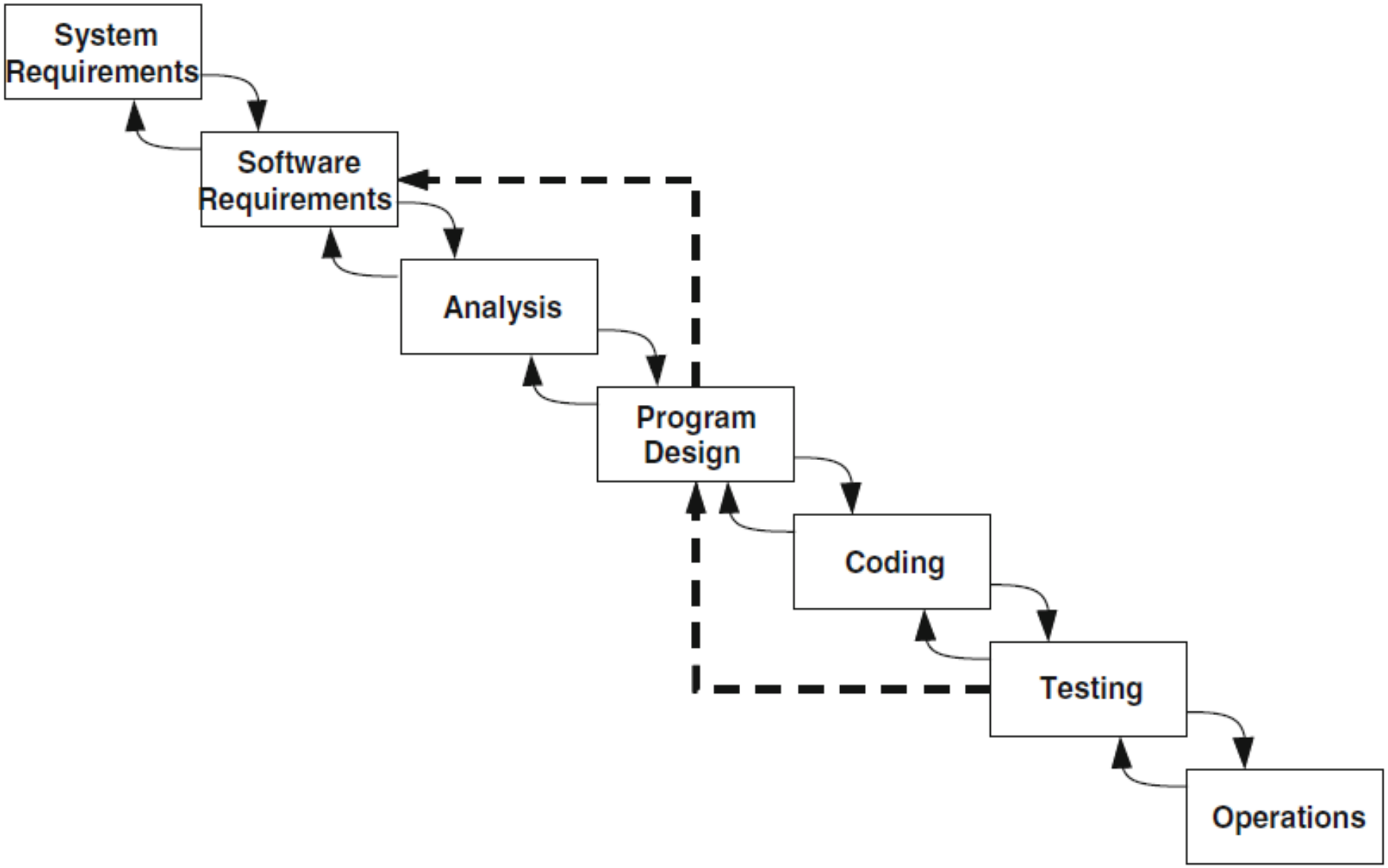


Fig. 2.3 Waterfall model as described by Royce, with iterations

Waterfall Process Model

- Adhering to the sequential order of the activities is extremely hard to achieve, even if interaction with the immediate neighbouring activities is allowed.

- Necessary prerequisites to applying a prototyping approach include being familiar with the domain, methods, techniques, tools, engineering processes and having a very good understanding of the requirements.

- The requirements themselves must be stable, and one must possess high capabilities for effort estimation.

Waterfall Process Model

- **Advantages.**
- Typically, waterfall-like projects face only few problems during integration.



- Version and configuration management is also simplified.

Waterfall Process Model

- **Challenges.**
- Unexpected requirements or context changes pose high risks.
- Gaining experience and learning during a project is difficult.
- Fixed delivery deadlines are risky for waterfall projects.
- The documentation created is often voluminous and heavyweight.
- The waterfall approach does not scale very well for large projects and long cycle times.

Waterfall Process Model

- The waterfall process model is often referenced, but rarely applied in its strict form.

The Iterative Enhancement Model

- Basili and Turner described an iterative approach to developing software systems in1975

- The iterative enhancement model proposes to first implement a (properly chosen) part of the complete system, and then add functionality in a number of iterations, which all together finally form the complete system.

- Each iteration is completed in a waterfall-like style, i.e., a requirements analysis for the respective iteration is followed by designing the system part of the iteration, which is again followed by its implementation, and so on.

The Iterative Enhancement Model

- The result of each iteration is integrated with the already existing system.
- The focus of an iteration may be on introducing new functionality, but it might also be on refinement, improvement, or architectural consolidation.
- A prerequisite for applying the iterative enhancement model is that the problem permits incremental development.

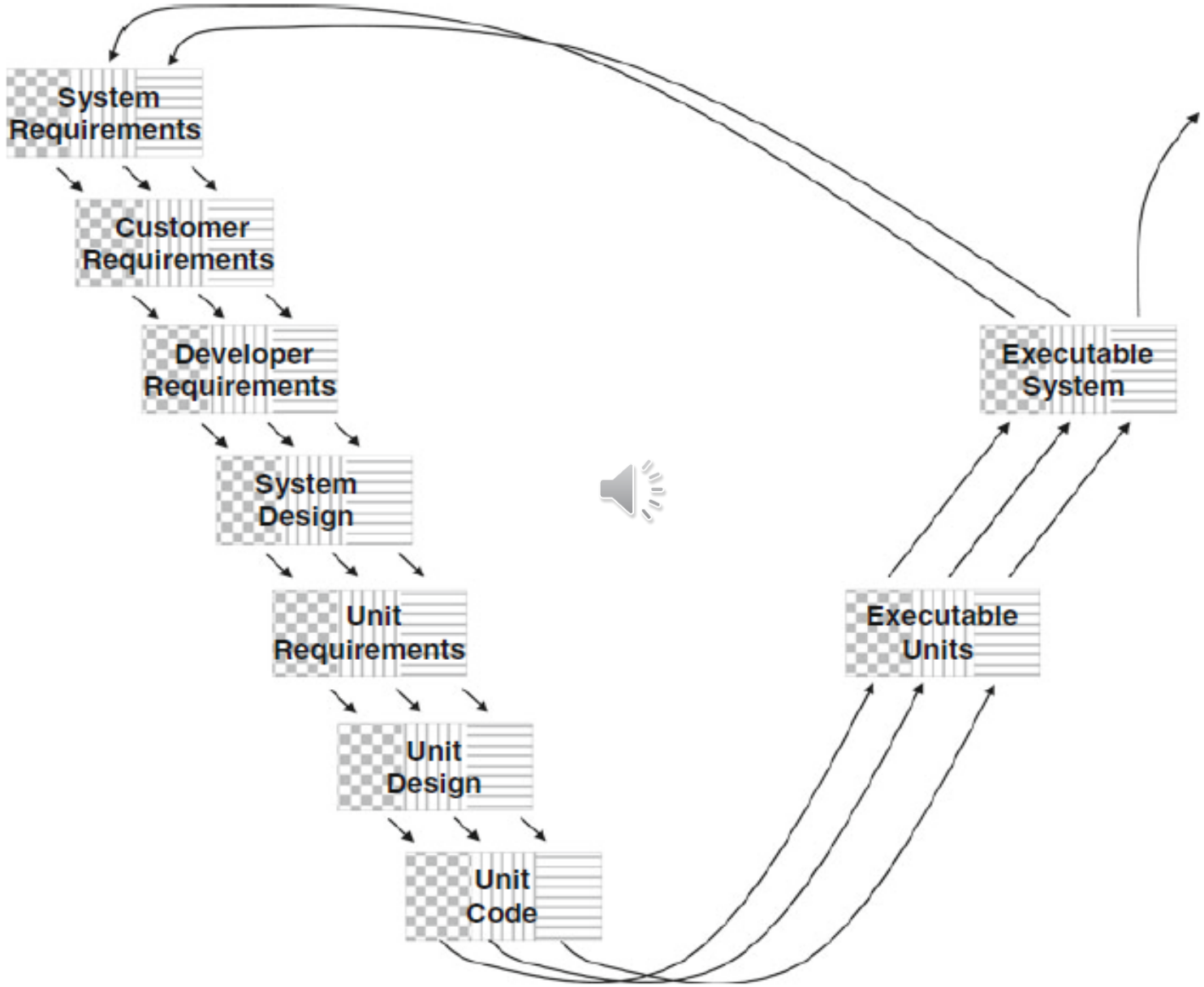


Fig. 2.4 Iterative enhancement model (three iterations)

The Iterative Enhancement Model

- **Advantages.**
- Iterative projects support efficient learning. With iterations designed properly, the core of the final product is available very early, thus featuring essential properties of the complete product.
- This allows for early customer involvement and feedback.
- Iteratively developing the requirements helps when requirements are not entirely clear or still unstable.

The Iterative Enhancement Model

- Integration testing is supported due to relatively small increments being added at a time.

- In case of fixed delivery dates, incremental development helps to ensure that the most important functionality can actually be delivered—and the customer can decide what is most important to him.

The Iterative Enhancement Model

- **Challenges.**
- Since the product design is based on the current set of requirements, there is a risk that requirements showing up later may be expensive to fulfil, due to design decisions made earlier.

- Good and comprehensive version and configuration management is necessary to distinguish increments.

The Iterative Enhancement Model

- Integration may become increasingly difficult with the number of iterations, depending on how well requirements may be partitioned and on the system architecture.

- Many modern software process models, such as:
- The Unified Process,
- Extreme Programming,
- Scrum
- follow an iterative pattern.